HY-225 ex09

Σαράντης Σαραντινός csd4511

9.9)

1)

- Instruction Fetch: Υπολογίζεται η επόμενη εντολή, η οποία είναι είτε η από κάτω (pc = pc + 4), είτε λαμβάνεται από κάποιο άλμα.
- Instruction Decode: Η εντολή αποκωδικοποιείται ερμηνεύοντας τον opcode και λαμβάνοντας τους καταχωρητές και τις σταθερές, αποφασίζεται ποια πράξη απαιτείται να συμβεί και αναλόγως στέλνονται σήματα στις υπόλοιπες βαθμίδες pipeline.
- Execute: Εκτελεί την λειτουργία που έχει καθοριστεί, η οποία μπορεί να περιλαμβάνει αριθμητικές και λογικές πράξεις σε δεδομένα καταχωρητών, υπολογισμό διευθύνσεων για λειτουργίες μνήμης ή προσδιορισμό διευθύνσης στόχου για περίπτωση διακλάδωσης, ανάγνωση τιμών από αρχείο καταχωρητή.
- Memory Access: Πρόσβαση στη μνήμη για ανάγνωση ή εγγραφή, μόνο σε περιπτώσεις που το απαιτούν.
- Write Back: Γράφει το αποτέλεσμα σε καταχωρητή προορισμού, σε περίπτωση που απαιτείται.

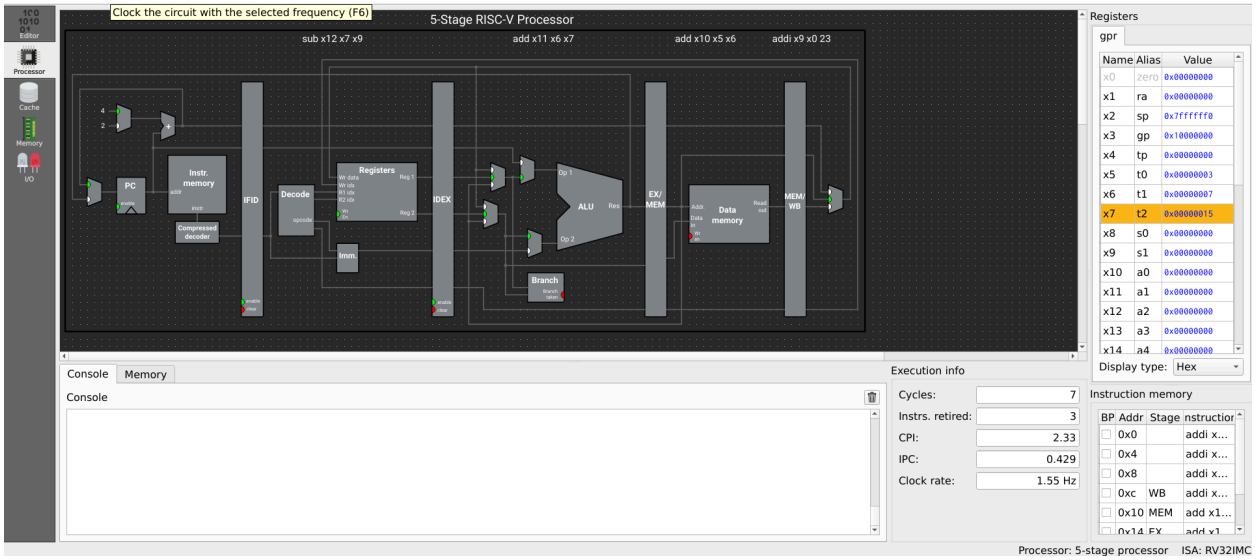
2) Οι απαραίτητες βαθμίδες είναι οι παρακάτω:
Instruction Fetch η οποία είναι αναγκαία για τον υπολογισμό της επόμενης εντολής, διαφορετικά δεν υπάρχει ροή,
Η Instruction Decode καθώς χωρίς αποκωδικοποίηση εντολής δεν μπορούμε να ξέρουμε τι (πράξη) πρέπει να συμβεί,
Και η Execute καθώς δίχως αυτήν δεν μπορούμε να προβούμε στην εκτέλεση των πράξεων που υποδεικνύουν οι εντολές (εκτός από εξαιρέσεις όπως συγκεκριμένα άλματα και branches που προσπερνούν την execute βαθμίδα).

3) Για τις εντολές αριθμητικών πράξεων δεν κάνει κάτι χρήσιμο η βαθμίδα Memory Access καθώς δεν χρειάζονται πρόσβαση στη μνήμη εφόσον εκτελούν μόνο πράξεις.
Για τις εντολές store δεν κάνει κάτι χρήσιμο η βαθμίδα Write Back διότι δεν γράφουν σε καταχωρητές.

4)

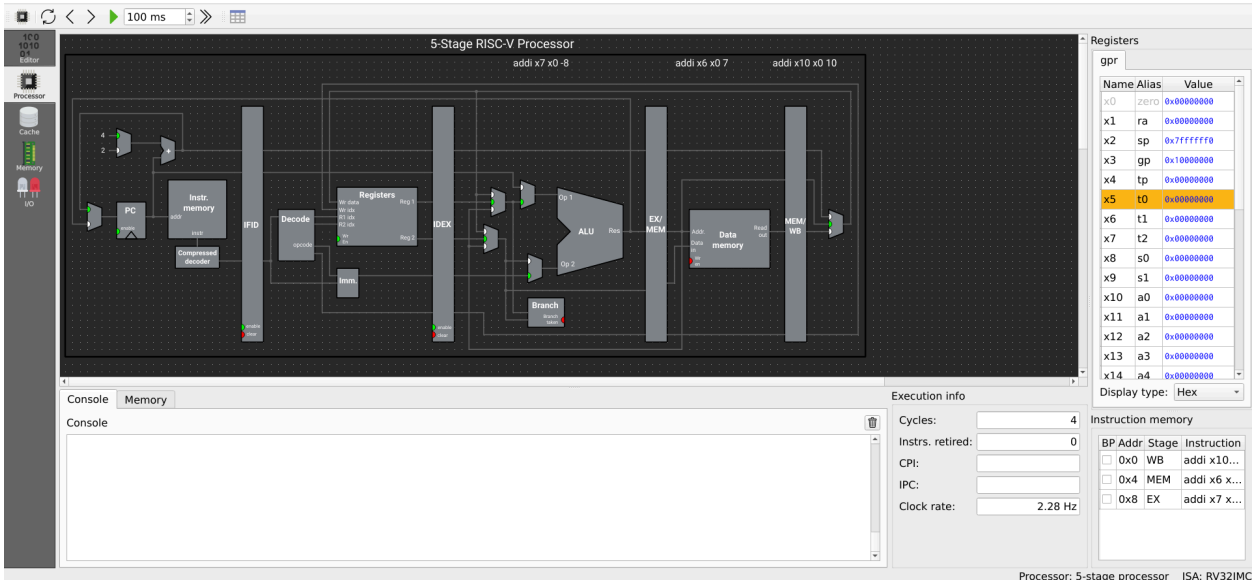
α) Αριθμητικές πράξεις μεταξύ καταχωρητών

```
# 9.9 4a
.text
#initialization
addi   x5, x0, 3
addi   x6, x0, 7
addi   x7, x0, 21
addi   x9, x0, 23
#arithmetic operations
add    x10, x5, x6
add    x11, x6, x7
sub    x12, x7, x9
```



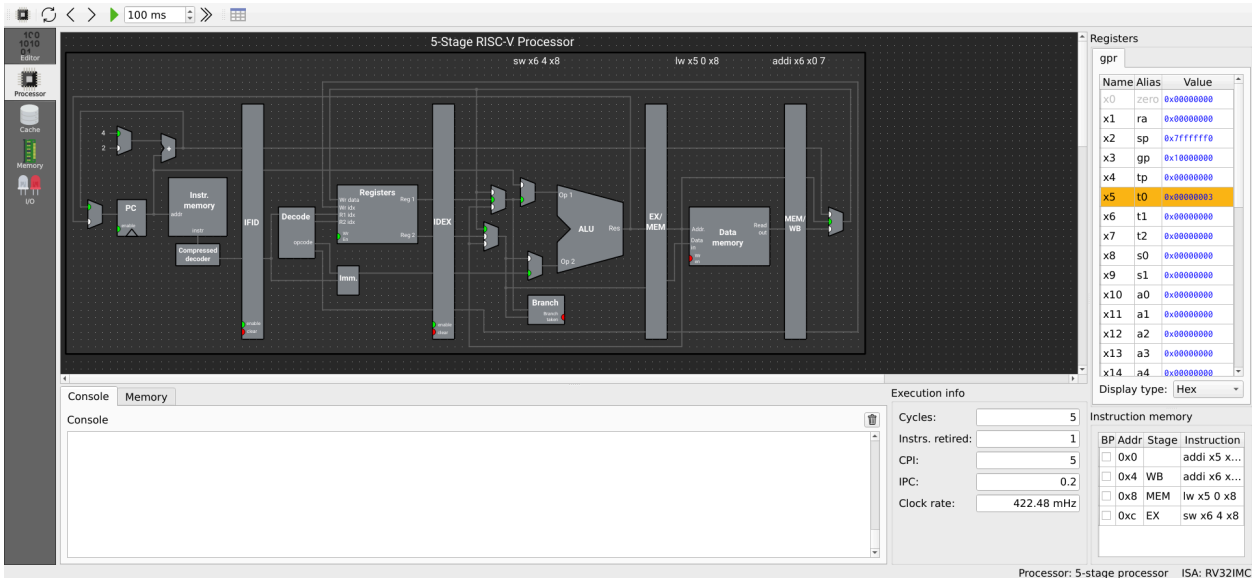
β) Πράξεις μεταξύ καταχωρητών και σταθερών

```
# 9.9 4b
.text
addi  x10, x0, 10
addi  x6, x0, 7
addi  x7, x0, -8
```



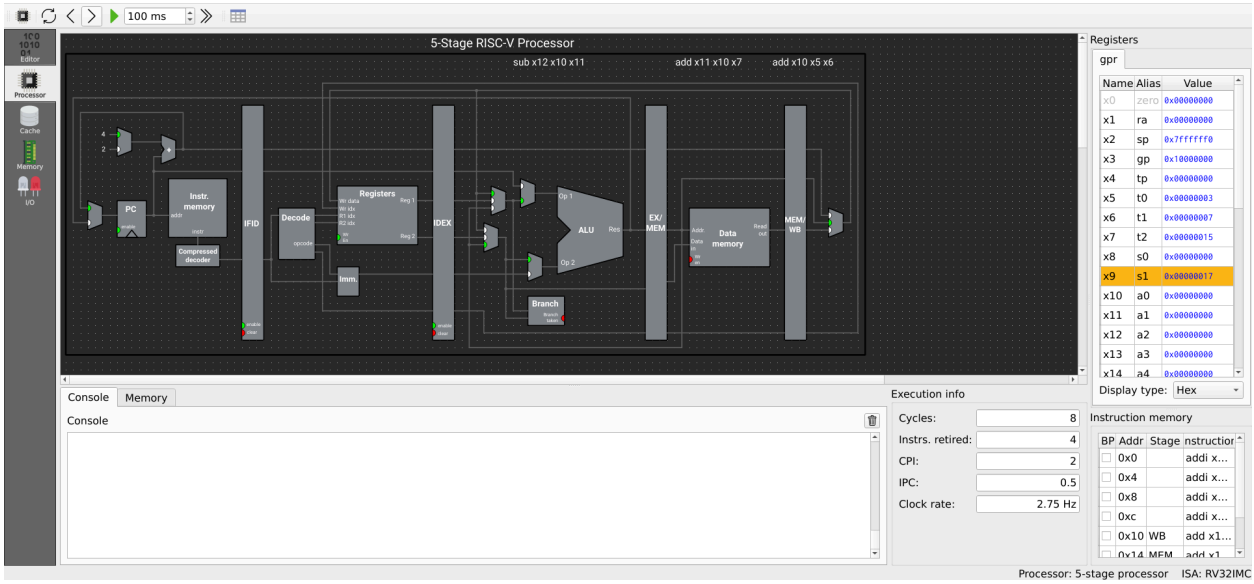
γ) Εντολές load και store

```
# 9.9 4c
.text
#initiazitation
addi  x5, x0, 3
addi  x6, x0, 7
#load and store commands
lw    x5, 0(x8)
sw    x6, 4(x8)
```



5) Οι εντολές αλλάζουν ως εξής:

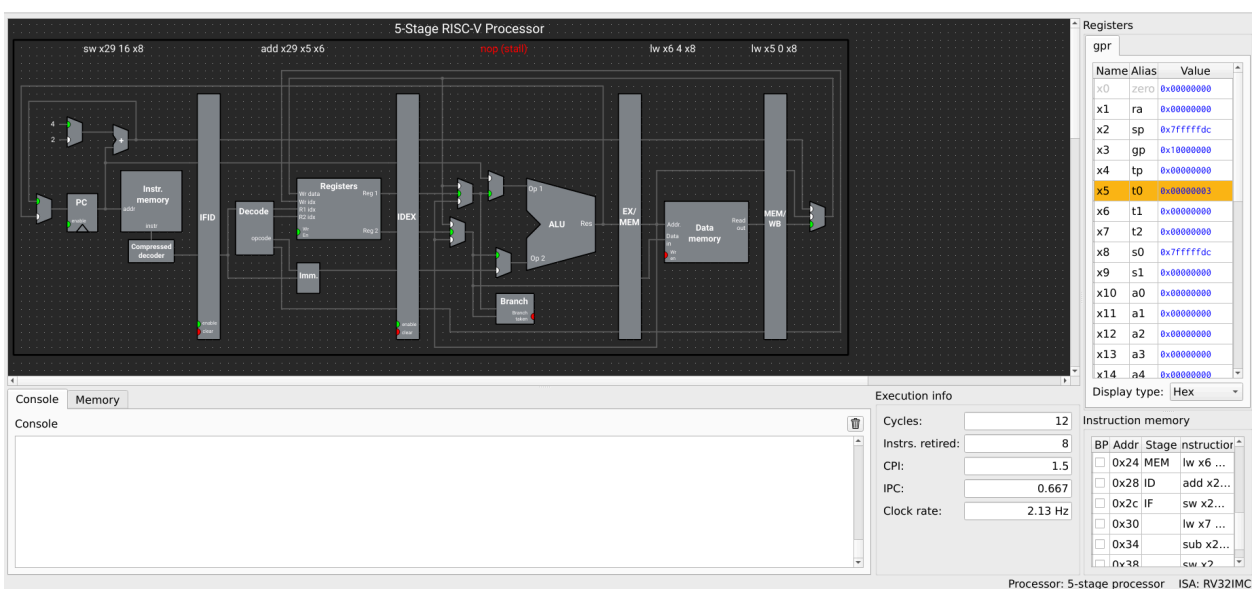
```
# 9.9 5
.text
#initialization
addi  x5, x0, 3
addi  x6, x0, 7
addi  x7, x0, 21
addi  x9, x0, 23
#arithmetic operations
add   x10, x5, x6
add   x11, x10, x7
sub   x12, x10, x11
```



```
6).text
main:
        #allocate space for 5 words/integers 5*4 = 20
addi    sp, sp, -20
add     s0, x0, sp
        #initiazization
addi    t0, x0, 1
sw      t0, 0(s0) #a = 1
addi    t0, x0, 2
sw      t0, 4(s0) #b = 2
addi    t0, x0, 3
sw      t0, 8(s0) #c = 3

lw      t0, 0(s0) #a
lw      t1, 4(s0) #b
add     t4, t0, t1
sw      t4, 16(s0) #f = a + b;

lw      t2, 8(s0) #c
sub     t3, t0, t2
sw      t3, 12(s0) #e = a - c;
```

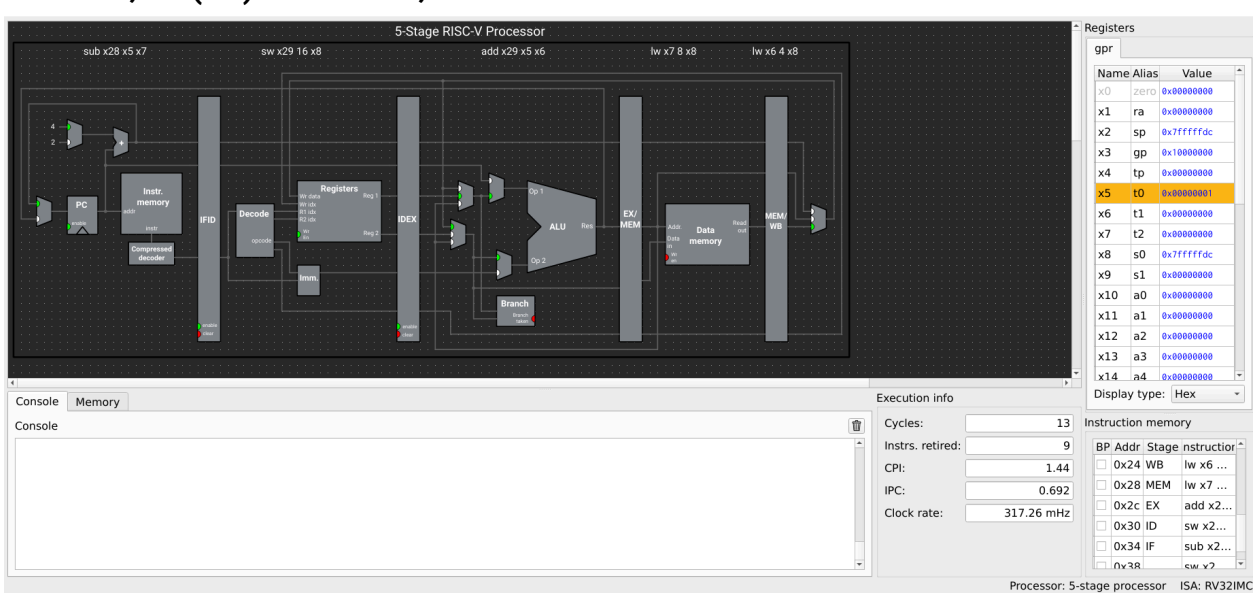


Έπειτα από αναδιάταξη λόγω καθυστερήσεων:

```
.text
main:
        #allocate space for 5 words/integers 5*4 = 20
addi    sp, sp, -20
add     s0, x0, sp
        #initiazization
addi    t0, x0, 1
sw      t0, 0(s0) #a = 1
addi    t0, x0, 2
sw      t0, 4(s0) #b = 2
addi    t0, x0, 3
sw      t0, 8(s0) #c = 3

lw      t0, 0(s0) #a
lw      t1, 4(s0) #b
lw      t2, 8(s0) #c

add     t4, t0, t1
sw      t4, 16(s0) #f = a + b;
sub     t3, t0, t2
sw      t3, 12(s0) #e = a - c;
```



7) Οι περισσότεροι από τους “χαμένους” κύκλους οφείλονται σε επιπλοκές εντολών, pipeline hazards, όπου γειτονικές εντολές εξαρτώνται η μία από τα αποτελέσματα της άλλης με αποτέλεσμα η δεύτερη να μπαίνει σε αναμονή μέχρις ότου να ολοκληρωθεί η πρώτη. Άλλοι δευτερεύοντες λόγοι μπορεί να είναι branches των οποίων ο προορισμός δεν είναι άμεσα γνωστός οπότε μέχρι να προσδιοριστεί το αποτέλεσμα οδηγούμαστε σε pipeline stall ή όταν πολλαπλές εντολές επιδιώκουν πρόσβαση σε συγκεκριμένο memory unit για fetch ή/και data load/store.